

Промежуточный отчёт по лабораторной работе №1 по предмету “Технологии управления данными”

Текст задачи:

Лабораторная работа №1. Разработка базы данных «Филиал» как источника данных.

В данной лабораторной работе требуется разработать прототип базы данных филиала торгового предприятия. База данных будет использована в дальнейшем как источник данных для наполнения хранилища данных.

Разрабатываемый прототип должен обеспечить хранение информации о следующих основных сущностях:

1. Покупатель;
2. Товар;
3. Сделка.

Информацию о покупателях можно ограничить уникальным идентификатором, а так же названием покупателя для обеспечения удобства работы с информацией.

Сущность Товар должна содержать следующие минимальные поля: уникальный номер товара, название товара, цена товара по каталогу. Следует отметить что каждый товар принадлежит к одной или нескольким категориям. Таким образом следует создать дополнительную сущность Категория, и обеспечить связь многие ко многим с сущностью Товар.

Сущность Сделка должна отражать информацию о факте покупки товара покупателем. Требуется отметить, что покупатель за одну покупку может купить несколько наименований товара в разном количестве, а также информацию о цене, за которую был куплен товар (она может отличаться от заявленной цены, например с учетом скидки), а также общую сумму сделки.

Целесообразно сделать связь между сущностями Сделка и Товар через промежуточную сущность содержащую информацию о количестве купленного товара и цена. Сущность Сделка будет содержать итоговую сумму покупки, а также дополнительную информацию, например дату совершения сделки.

На основании имеющейся информации необходимо создать реляционную модель данных, и получить утвердить модель у преподавателя.

На основании имеющейся реляционной модели создать даталогическую модель, в соответствии с требованиями типов данных, используемых на конкретной версии SQL Server, и так же утвердить у преподавателя.

На основании данных моделей создать скрипт создания базы данных, с учетом следующих требований:

1. Каждая таблица должна иметь суррогатный целочисленный первичный ключ;
2. Каждая таблица, помимо атрибутов необходимых для хранения данных в соответствии с заданиями, должны содержать атрибут rowguid – уникальный идентификатор в рамках всей системы, и ModifiedDate – дата/время добавления или последней модификации строки;
3. Все связи должны быть реализованы как дополнительные изменения таблицы после их создания;
4. База данных должна содержать минимально необходимое количество ограничений и триггеров необходимых для стабильной работы;
5. Рекомендовано использовать английский язык для названий сущностей, атрибутов и других объектов БД.

Проверить работоспособность созданного скрипта и убедиться в стабильности его работы.

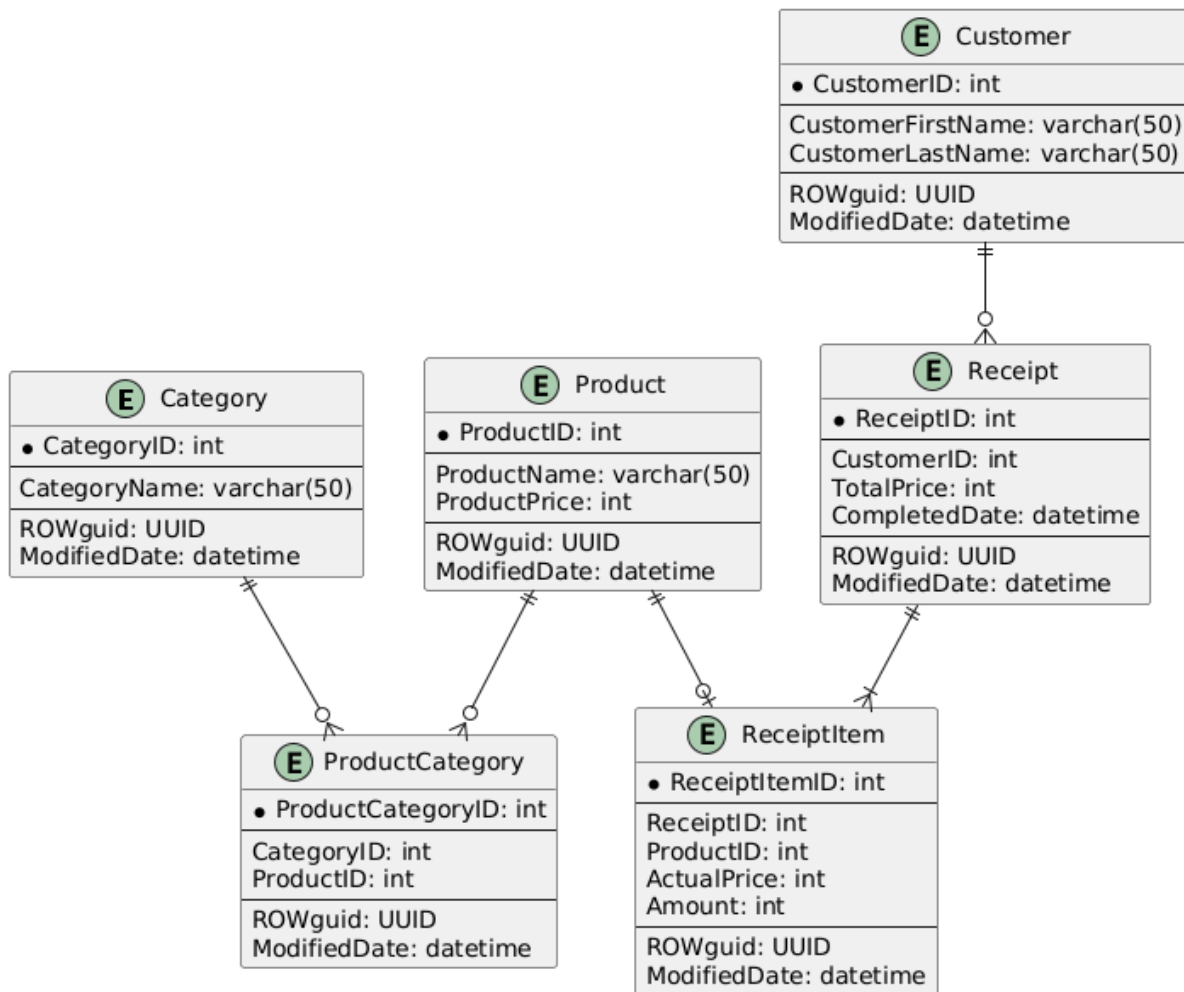
Подготовить два комплекта операций вставки в базу данных, обеспечивающих наполнение базы необходимыми для тестирования данными. Объем данных должен быть минимум двадцать пять строк для базовых сущностей, и минимум пятьдесят строк для сущностей, реализующих связь.

Создать две базы данных по разработанному скрипту, и наполнить соответствующим набором данных. Созданные базы данных обозначить как «Филиал Запад» и «Филиал Восток». При создании скрипта наполнения данными рекомендуется использовать ИИ. Выбор реляционной СУБД остается за студентом. СУБД, и полученный набор скриптов необходимо согласовать с преподавателем.

Решение:

Выбранная реляционная СУБД - PostgreSQL.

Диаграмма связей сущностей:



<Entity>ID - целочисленный первичный ключ сущности.

ROWguid - уникальный идентификатор строки (по условию).

ModifiedDate - дата модификации строки (по условию).

Category - сущность "Категория", содержит также название категории.

Product - сущность "Товар", содержит название товара и цену товара.

ProductCategory - сущность "Категория товара", обеспечивает связь многие-ко-многим между Товаром и Категорией

Customer - сущность "Покупатель", содержит имя и фамилию покупателя.

Receipt - сущность "Сделка" (чек), содержит ID покупателя, полную цену чека, дату совершения сделки.

ReceiptItem - сущность "Товар в сделке" (предмет/колонка в чеке) - сущность, обеспечивающая связь между Товаром и Сделкой, содержит количество товара и по какой цене он был куплен.

Скрипты и код:

 entities_creation.sql

 seeding.py

entities_creation.sql - скрипт создания базы данных (в соответствии по диаграмме и описанию выше):

```
CREATE TABLE IF NOT EXISTS Category (  
    ROWguid UUID DEFAULT gen_random_uuid(),  
    ModifiedDate TIMESTAMP DEFAULT now(),  
    CategoryID INTEGER PRIMARY KEY,  
    CategoryName VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE IF NOT EXISTS Customer (  
    ROWguid UUID DEFAULT gen_random_uuid(),  
    ModifiedDate TIMESTAMP DEFAULT now(),  
    CustomerID INTEGER PRIMARY KEY,  
    CustomerFirstName VARCHAR(50) NOT NULL,  
    CustomerLastName VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE IF NOT EXISTS Product (  
    ROWguid UUID DEFAULT gen_random_uuid(),  
    ModifiedDate TIMESTAMP DEFAULT now(),  
    ProductID INTEGER PRIMARY KEY,  
    ProductName VARCHAR(50) NOT NULL,  
    ProductPrice INTEGER NOT NULL  
);  
  
CREATE TABLE IF NOT EXISTS ProductCategory (  
    ROWguid UUID DEFAULT gen_random_uuid(),  
    ModifiedDate TIMESTAMP DEFAULT now(),  
    ProductCategoryID INTEGER PRIMARY KEY,  
    CategoryID INTEGER,  
    ProductID INTEGER,  
  
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID),  
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)  
);  
  
CREATE TABLE IF NOT EXISTS Receipt (  
    ROWguid UUID DEFAULT gen_random_uuid(),  
    ModifiedDate TIMESTAMP DEFAULT now(),  
    ReceiptID INTEGER PRIMARY KEY,  
    CustomerID INTEGER NOT NULL,  
    TotalPrice INTEGER NOT NULL,  
    CompletedDate TIMESTAMP NOT NULL,
```

```

    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);

CREATE TABLE IF NOT EXISTS ReceiptItem (
    ROWguid UUID DEFAULT gen_random_uuid(),
    ModifiedDate TIMESTAMP DEFAULT now(),
    ReceiptItemID INTEGER PRIMARY KEY,
    ReceiptID INTEGER NOT NULL,
    ProductID INTEGER NOT NULL,
    ActualPrice INTEGER NOT NULL,
    Amount INTEGER NOT NULL,

    FOREIGN KEY (ReceiptID) REFERENCES Receipt(ReceiptID),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);

```

seeding.py - скрипт наполнения баз данных, написанный на языке Python:

```

import psycopg2
import random
import datetime
import uuid

def random_datetime():
    start = datetime.datetime.strptime('1997-04-01 19:00', '%Y-%m-%d %H:%M')
    end = datetime.datetime.strptime('2023-03-24 20:00', '%Y-%m-%d %H:%M')
    delta = end - start
    int_delta = (delta.days * 24 * 60 * 60) + delta.seconds
    random_second = random.randrange(int_delta)
    return str(start + datetime.timedelta(seconds=random_second))

def gen_uuid():
    return uuid.uuid1()

db_config_west = {
    "dbname": "west_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5446"
}

db_config_east = {
    "dbname": "east_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5447"
}

```

```

conn_west = psycopg2.connect(**db_config_west)
cur_west = conn_west.cursor()
conn_east = psycopg2.connect(**db_config_east)
cur_east = conn_east.cursor()

categories = ["fruits", "beverages", "electronics", "vegetables", "dairy"]
for ID in range(len(categories)):
    cur_west.execute("INSERT INTO Category (ROWguid, CategoryID,
CategoryName) VALUES (%s, %s, %s)",
                      (str(gen_uuid()), ID, categories[ID]))
    cur_east.execute("INSERT INTO Category (ROWguid, CategoryID,
CategoryName) VALUES (%s, %s, %s)",
                      (str(gen_uuid()), ID, categories[ID]))

customers = ["Adam", "Boris", "Charlie", "David", "Ethan",
             "Florian", "Gerald", "Hilbert", "Ian", "John",
             "Kevin", "Lucas", "Martin", "Noah", "Opal",
             "Peter", "Quinsy", "Roger", "Samuel", "Thomas",
             "Uriel", "Victor", "Wesley", "Xavier", "Yvonne",
             "Zachary"]
for ID in range(len(customers)):
    cur_west.execute("INSERT INTO Customer (ROWguid, CustomerID,
CustomerFirstName, CustomerLastName) VALUES (%s, %s, %s, %s)",
                      (str(gen_uuid()), ID, customers[ID], "West"))
    cur_east.execute("INSERT INTO Customer (ROWguid, CustomerID,
CustomerFirstName, CustomerLastName) VALUES (%s, %s, %s, %s)",
                      (str(gen_uuid()), ID, customers[ID], "East"))

products = [
    ["apple", "banana", "kiwi", "mango", "pear"],
    ["coffee", "cola", "pepsi", "tea", "water"],
    ["camera", "gaming console", "fryer", "TV", "vacuum cleaner"],
    ["cucumber", "garlic", "onion", "pumpkin", "tomato"],
    ["cheese", "cream", "milk", "milkshake", "yoghurt"]
]
product_prices = [
    [10, 7, 25, 20, 15],
    [30, 10, 12, 20, 5],
    [300, 450, 200, 700, 450],
    [5, 8, 7, 32, 10],
    [35, 45, 18, 42, 24]
]
for categoryID in range(len(categories)):
    for productIDinCaterogy in range(len(products[categoryID])):
        ID = categoryID * 5 + productIDinCaterogy
        cur_west.execute("INSERT INTO Product (ROWguid, ProductID,
ProductName, ProductPrice) VALUES (%s, %s, %s, %s)",
                          (str(gen_uuid()), ID,
products[categoryID][productIDinCaterogy],
product_prices[categoryID][productIDinCaterogy]))

```

```

cur_west.execute("INSERT INTO ProductCategory (ROWguid,
ProductCategoryID, CategoryID, ProductID) VALUES (%s, %s, %s, %s)",
                (str(gen_uuid()), ID, categoryID, ID))
cur_east.execute("INSERT INTO Product (ROWguid, ProductID,
ProductName, ProductPrice) VALUES (%s, %s, %s, %s)",
                (str(gen_uuid()), ID,
products[categoryID][productIDinCaterogy],
                product_prices[categoryID][productIDinCaterogy]))
cur_east.execute("INSERT INTO ProductCategory (ROWguid,
ProductCategoryID, CategoryID, ProductID) VALUES (%s, %s, %s, %s)",
                (str(gen_uuid()), ID, categoryID, ID))

customerIDs = [i for i in range(len(customers))]
random.shuffle(customerIDs)
receiptIDWest = 0
receiptItemIDWest = 0
receiptIDEast = 0
receiptItemIDEast = 0
parity = 0
for customerID in customerIDs:
    for receipt in range(random.randint(2, 8)):
        products_count = random.randint(1, 3)
        items = []
        total_price = 0
        completed_datetime = random_datetime()
        for _ in range(products_count):
            categoryID = random.randint(0, 4)
            productIDinCaterogy = random.randint(0, 4)
            productID = categoryID * 5 + productIDinCaterogy
            price = product_prices[categoryID][productIDinCaterogy]
            discount = random.randint(0, 20) # in %
            actual_price = price * (100 - discount) // 100
            if categoryID != 2:
                amount = random.randint(1, 12)
            else:
                amount = 1
            items.append([productID, actual_price, amount])
            total_price += actual_price * amount
        if parity == 0:
            cur_west.execute("INSERT INTO Receipt (ROWguid, ReceiptID,
CustomerID, TotalPrice, CompletedDate) "
                            "VALUES (%s, %s, %s, %s, %s)",
                            (str(gen_uuid()), receiptIDWest, customerID,
total_price, completed_datetime))
            receiptIDWest += 1
            for item in items:
                cur_west.execute("INSERT INTO ReceiptItem (ROWguid,
ReceiptItemID, ReceiptID, ProductID, ActualPrice, Amount) "
                                "VALUES (%s, %s, %s, %s, %s, %s)",
                                (str(gen_uuid()), receiptIDWest + item[2],
receiptIDWest, item[0], item[1], item[2]))
                receiptItemIDWest += 1
            parity = 1
        else:
            cur_east.execute("INSERT INTO Receipt (ROWguid, ReceiptID,
CustomerID, TotalPrice, CompletedDate) "
                            "VALUES (%s, %s, %s, %s, %s)",
                            (str(gen_uuid()), receiptIDEast, customerID,
total_price, completed_datetime))
            receiptIDEast += 1
            for item in items:
                cur_east.execute("INSERT INTO ReceiptItem (ROWguid,
ReceiptItemID, ReceiptID, ProductID, ActualPrice, Amount) "
                                "VALUES (%s, %s, %s, %s, %s, %s)",
                                (str(gen_uuid()), receiptIDEast + item[2],
receiptIDEast, item[0], item[1], item[2]))
                receiptItemIDEast += 1
            parity = 0

```

```

                                (str(gen_uuid()), receiptItemIDWest,
receiptIDWest, item[0], item[1], item[2]))
                                receiptItemIDWest += 1
                                receiptIDWest += 1
                                parity = 1
                                else:
                                    cur_east.execute("INSERT INTO Receipt (ROWguid, ReceiptID,
CustomerID, TotalPrice, CompletedDate) "
                                                    "VALUES (%s, %s, %s, %s, %s)",
                                                    (str(gen_uuid()), receiptIDEast, customerID,
total_price, completed_datetime))
                                    for item in items:
                                        cur_east.execute("INSERT INTO ReceiptItem (ROWguid,
ReceiptItemID, ReceiptID, ProductID, ActualPrice, Amount)"
                                                        "VALUES (%s, %s, %s, %s, %s, %s)",
                                                        (str(gen_uuid()), receiptItemIDEast,
receiptIDEast, item[0], item[1], item[2]))
                                        receiptItemIDEast += 1
                                        receiptIDEast += 1
                                        parity = 0

conn_west.commit()
conn_east.commit()
cur_west.close()
cur_east.close()
conn_west.close()
conn_east.close()
print("All done!")

```